

Automate your NX-OS Data Center fabrics using Ansible, NETCONF and YANG

Gerard Sheehan
Product Manager



Webex App

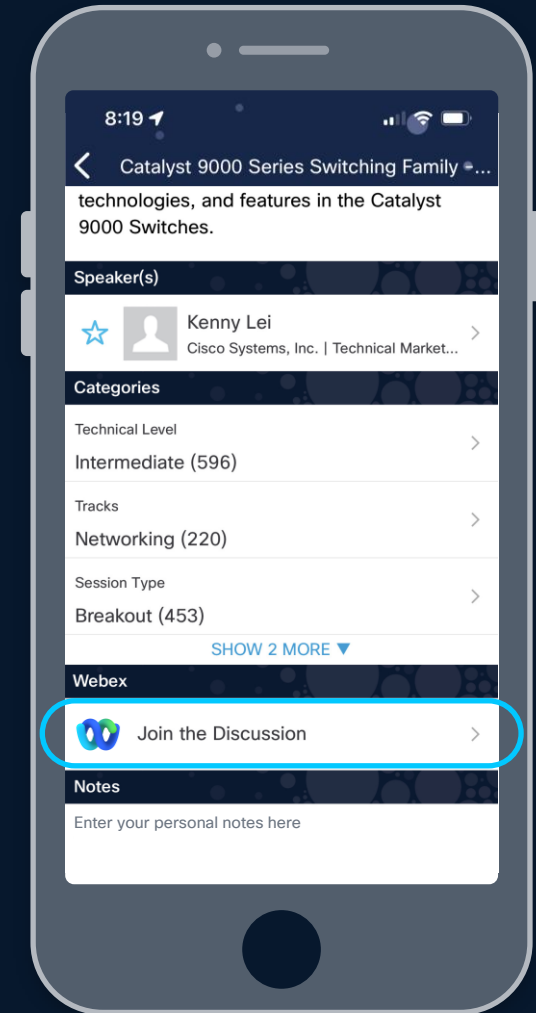
Questions?

Use Webex app to chat with the speaker after the session

How

- 1 Find this session in the Cisco Events app
- 2 Click “Join the Discussion”
- 3 Install the Webex app or go directly to the Webex space
- 4 Enter messages/questions in the Webex space

Webex spaces will be moderated by the speaker until February 27, 2026.

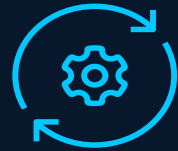
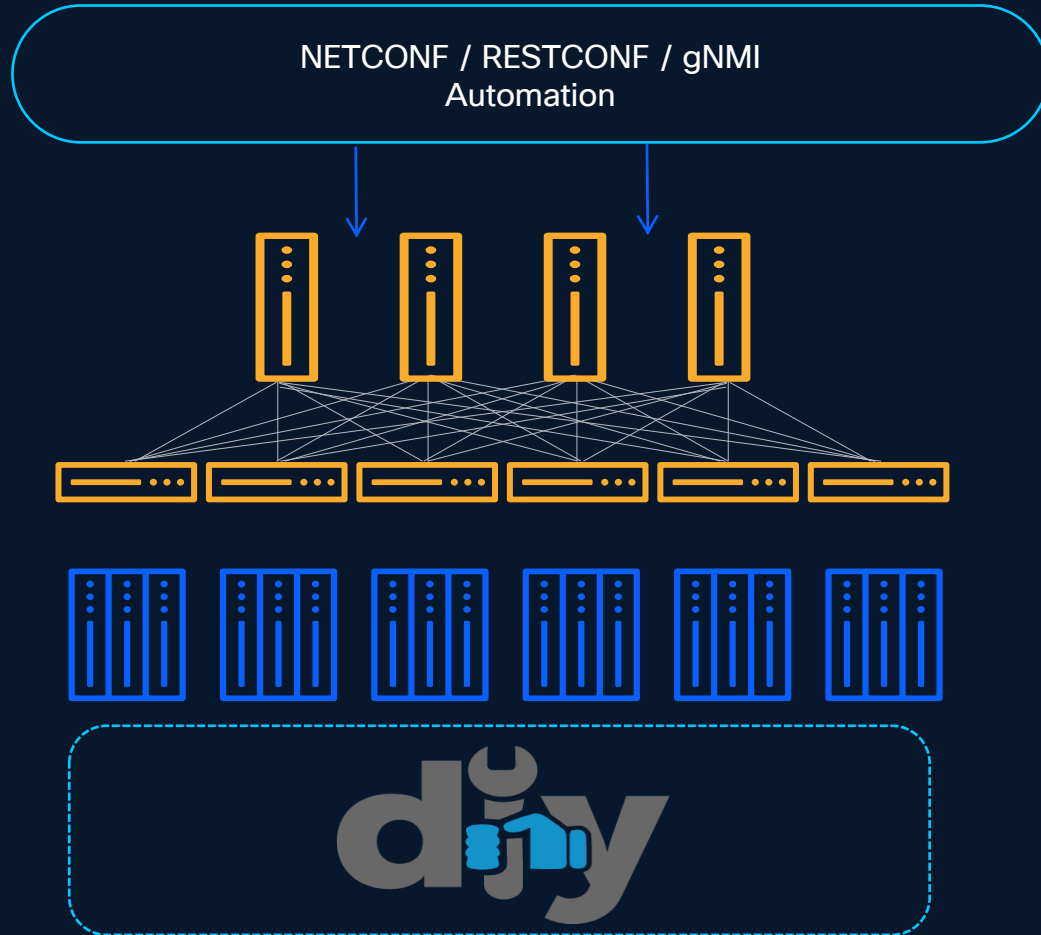


Agenda

- 01 Introduction
- 02 API Overview
- 03 Tying it All Together
- 04 Working with YANG
- 05 Demo
- 06 Conclusion

Introduction

Why Use Open Tools



Operational Control

- Enhanced automation flexibility
- Inbuilt mechanisms to prevent inconsistent state



Operational Flexibility

- BYO Controller
- Leverage 3rd party commercial tools



Operational Simplicity

- Consistent “look and feel” across platforms
- Industry standard transport and encoding

Going Beyond Provisioning

Pre-Change Validation

- Verify that the defined data adheres to defined schema definition
- Integrate rules that enforce either technical or policy requirements in the configuration

Technical Review

- Use software discipline and development processes to ensure multi-engineer reviews of proposed changes

Automation Engine

- Automation is executed from central infrastructure
- Changes in declared configuration model force the execution of automation

Post-Change Automated Testing

- Testing to validate the applied configuration, the health and expected operational state of the environment

Types of Data Models



Controller Centric Data Models

Having a central touch point providing an API makes it the prime candidate



Solution Centric Data Models

Implementing a single model of a solution to work across different products

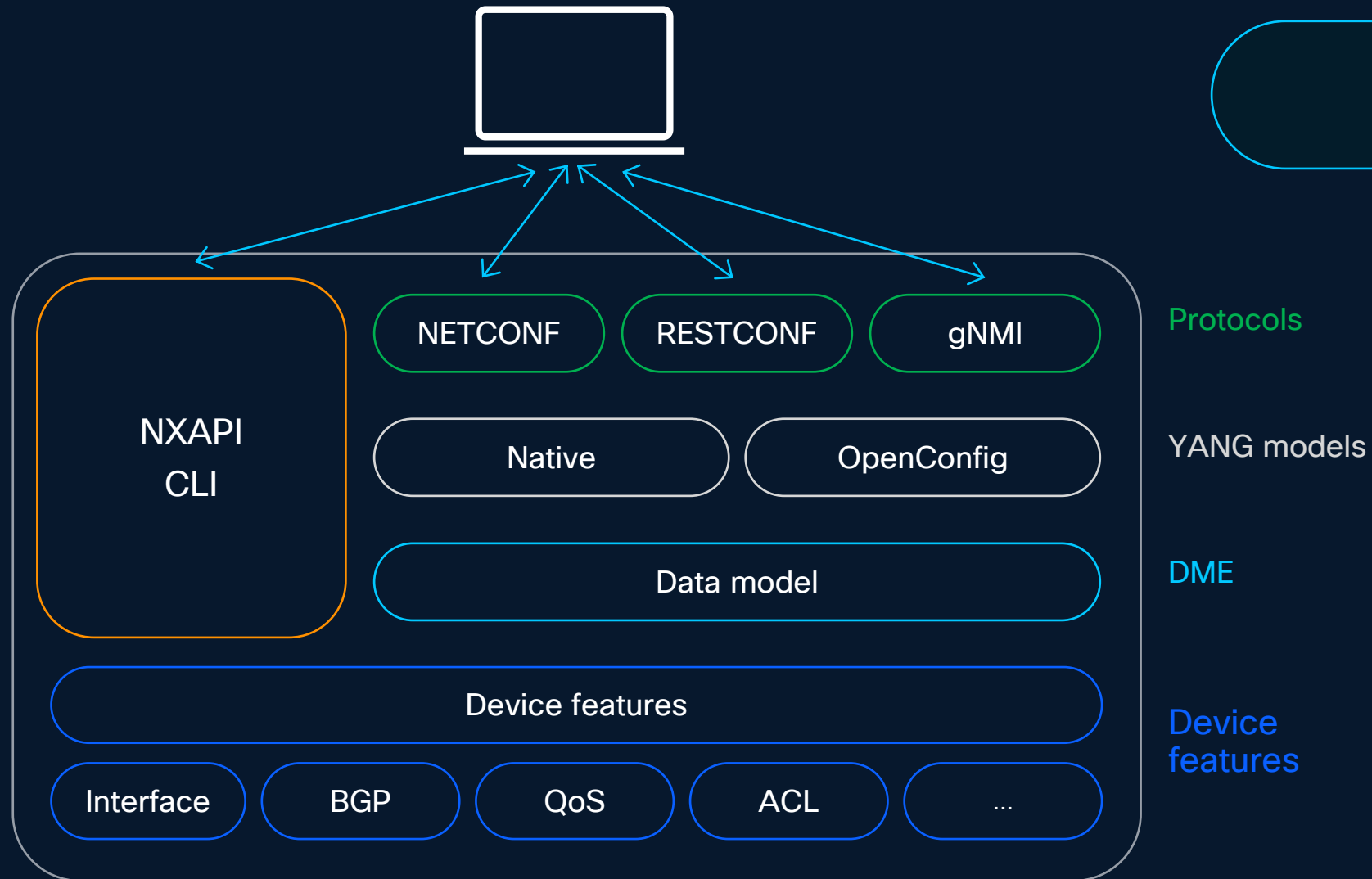


Device Centric Data Models

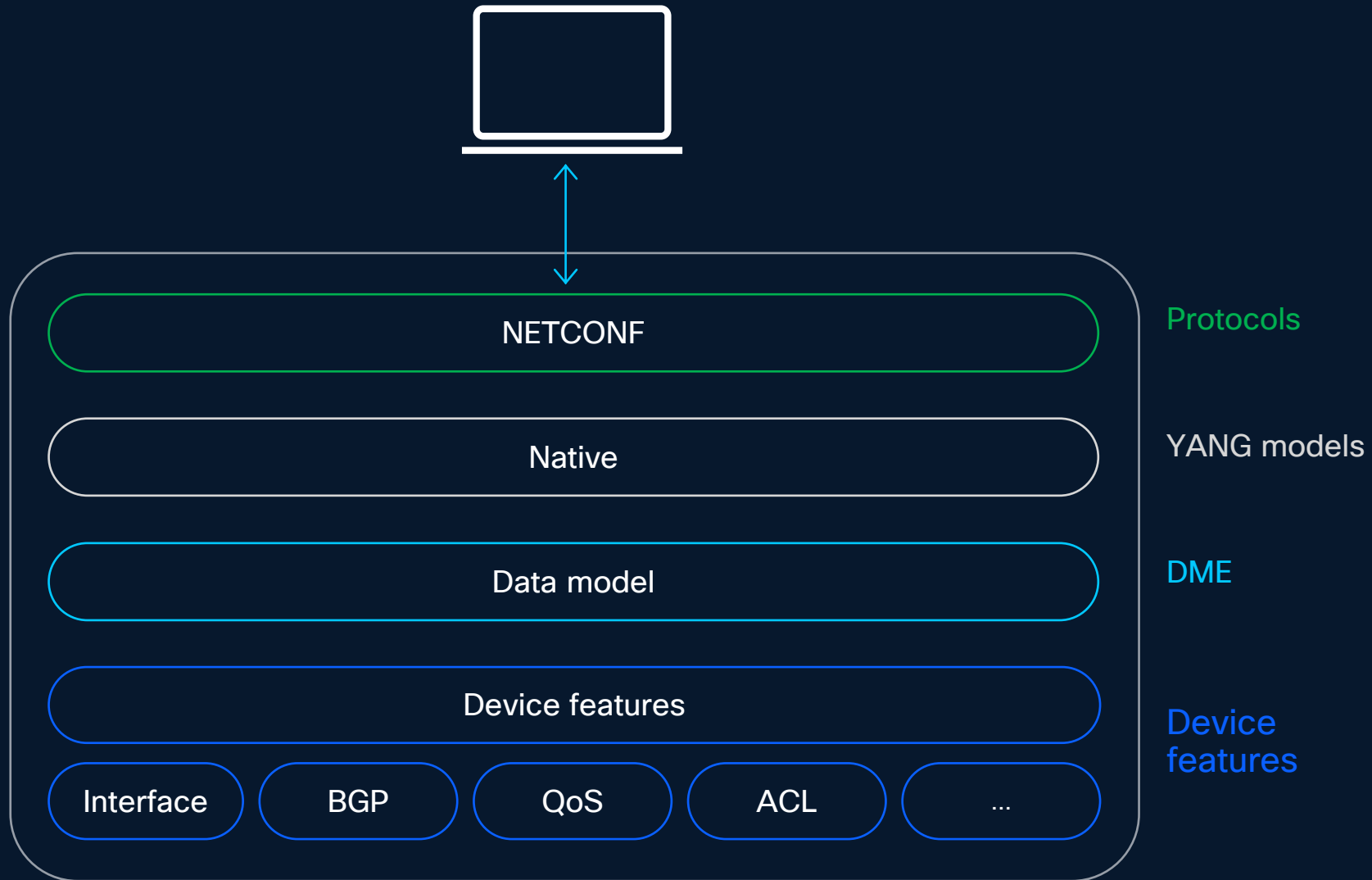
Replicate the same approach to model a set of device configurations

API Overview

Open Programmable APIs



NETCONF & Native YANG



NETCONF Capabilities

Operations supported

- get
- get-config
- edit-config
- copy-config
- lock/unlock
- validate
- Commit
- Commit w/ timeout

Edit config operations

- Merge: Merge payload w/ datastore
- Replace: Replace datastore w/payload
- Delete: Delete element from config
- Remove: Same as delete, except no error is return
- NETCONF trim
 - Return config without default values

Working with YANG

Pyang

An extensible YANG (RFC 6020/7950) validator. Provides a framework for plugins that can convert YANG modules to other formats.

<https://github.com/YangModels/yang/tree/main/vendor/cisco/nx>

```
gsheehan ~/workspace/devnet-1677/10.5-2  
➔ pyang -f tree openconfig-interfaces.yang
```

module: openconfig-interfaces

+--rw interfaces

+--rw interface* [name]

+--rw name -> ../config/name

+--rw config

| +--rw name? string

| +--rw type identityref

| +--rw mtu? uint16

| +--rw loopback-mode? boolean

| +--rw description? string

| +--rw enabled? boolean

+--ro state

| +--ro name? string

| +--ro type identityref

| +--ro mtu? uint16

| +--ro loopback-mode? boolean

| +--ro description? string

| +--ro enabled? boolean

| +--ro ifindex? uint32

| +--ro admin-status enumeration

Pyang

Modules YANG structures data models into modules and submodules

Containers group related nodes in subtree

Lists a sequence of list entries

Leaf simple data node with one value

<https://github.com/YangModels/yang/tree/main/vendor/cisco/nx>

```
gsheehan ~/workspace/devnet-1677/10.4-1  
➔ pyang -f tree openconfig-interfaces.yang
```

```
module: openconfig-interfaces  
  +--rw interfaces  
    +--rw interface* [name]  
      +--rw name          -> ../config/name  
      +--rw config  
        | +--rw name?      string  
        | +--rw type       identityref  
        | +--rw mtu?       uint16  
        | +--rw loopback-mode? boolean  
        | +--rw description? string  
        | +--rw enabled?   boolean  
        +--ro state  
          | +--ro name?     string  
          | +--ro type      identityref  
          | +--ro mtu?      uint16  
          | +--ro loopback-mode? boolean  
          | +--ro description? string  
          | +--ro enabled?   boolean  
          | +--ro ifindex?   uint32  
          | +--ro admin-status enumeration
```

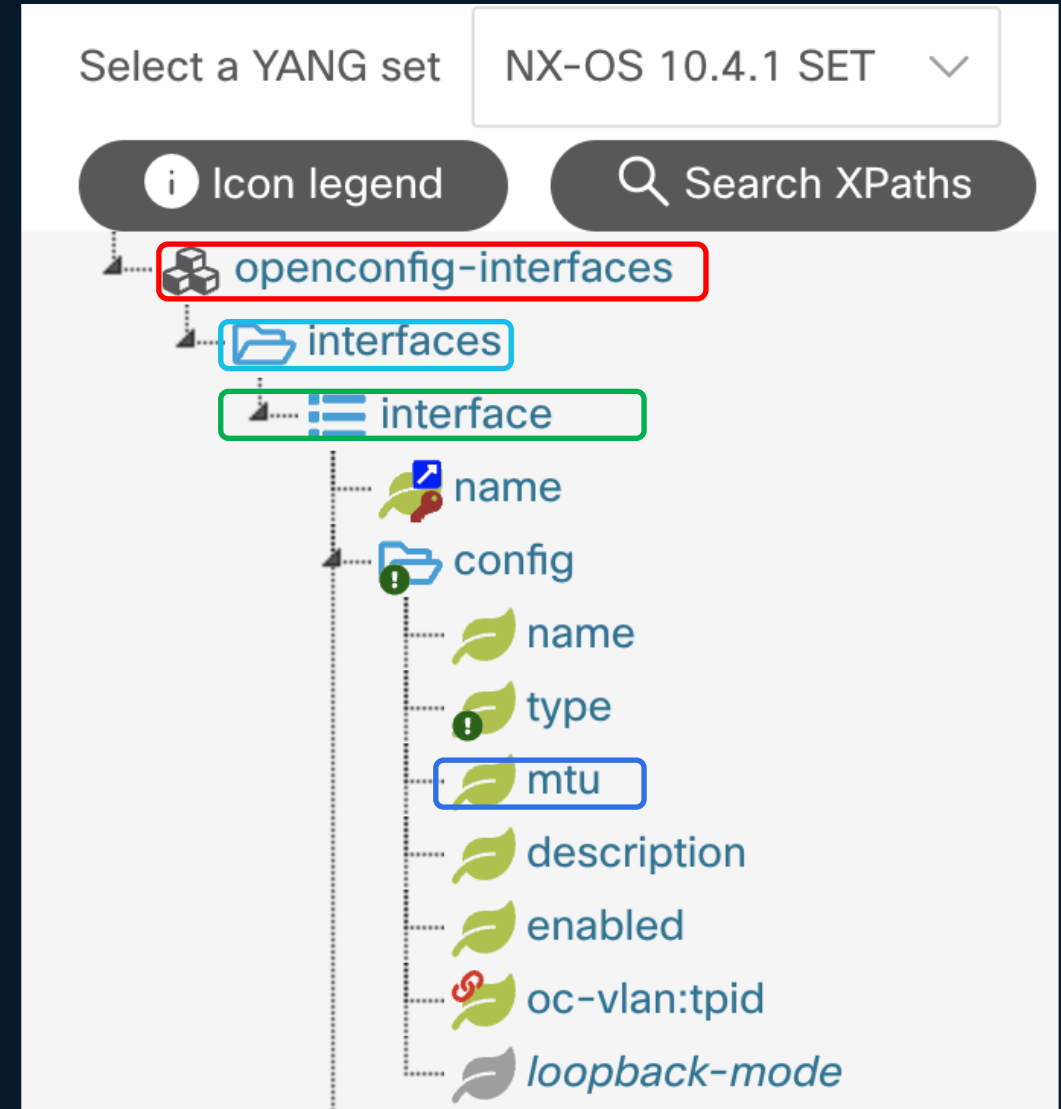
YANG Suite

Modules YANG structures data models into modules and submodules

Containers group related nodes in subtree

Lists a sequence of list entries

Leaf simple data node with one value



☒ Display schema nodes only ☐ Display all nodes

Node Properties

Name	mtu
Nodetype	leaf
Datatype	uint16
Description	Set the max transmission unit size in octets for the physical interface. If this is not set, the mtu is set to the operational default -- e.g., 1514 bytes on an Ethernet interface.
Module	openconfig-interfaces
Revision	2021-04-06
Xpath	/interfaces/interface/config/mtu
Prefix	oc-if
Namespace	http://openconfig.net/yang/interfaces
Schema Node Id	/interfaces/interface/config/mtu

NXAPI Sandbox

 **NX-API Sandbox** Username:admin Role:network-admin Version:10.2(4) Management ip:10.2.1.2 Hostname:leaf1

[NX-API References](#) [Yang Browser](#) [Yang Models](#) [Logout](#)

```
router bgp 64512
router-id 10.22.0.2
address-family l2vpn evpn
neighbor 10.22.0.2
remote-as 64512
update-source loopback0
address-family l2vpn evpn
send-community
send-community extended
vrf service
address-family ipv4 unicast
advertise l2vpn evpn
```

Method: RESTCONF (Yang)
Message format: json

POST restconf/data/Cisco-NX-OS-device:System/ Send Reset Convert

Request Python Python3 Java JavaScript Go-Lang

```
{
  "bgp-items": {
    "inst-items": {
      "asn": "64512",
      "dom-items": {
        "Dom-list": [
          {
            "name": "service",
            "af-items": {
              "DomAf-list": [
                {
                  "advertL2vpnEvpn": "enabled",
```

Response:

Copyright © 2014-2020 Cisco Systems, Inc. All rights reserved.

Tying it all together

Traditional CLI Method

Ansible

Variables



Jinja
Templates



CLI
Configuration

I3_interfaces:

- interface: eth1/1
ip_address: 10.1.1.1/31
mtu: 9216
description: "Link to Spine-1"
- interface: eth1/2
ip_address: 10.2.1.1/31
mtu: 9216
description: "Link to Spine-2"

```
{% for item in I3_interfaces %}  
interface {{ item.interface }}  
description {{ item.description }}  
ip address {{ item.ip_address }}  
mtu {{ item.mtu }}  
{% endfor %}
```

```
interface eth1/1  
description Link to Spine-1  
ip address 10.1.1.1/31  
mtu 9216  
interface eth1/2  
description Link to Spine-2  
ip address 10.2.1.1/31  
mtu 9216
```

YANG Phase 1: Building the Model

Ansible

Variables



Jinja
Templates



YANG model

I3_interfaces:

- interface: eth1/1
 - ip_address: 10.1.1.1/31
 - mtu: 9216
 - description: "Link to Spine-1"
- interface: eth1/2
 - ip_address: 10.2.1.1/31
 - mtu: 9216
 - description: "Link to Spine-2"

```
{% for item in I3_interfaces %}
<If-list>
<id>{{ item.interface }}</id>
<addr-items>
  <Addr-list>
    <addr>{{ item.ip_address }}</addr>
  </Addr-list>
</addr-items>
</If-list>
{% endfor %}
```

```
<if-items>
  <If-list>
    <id>eth1/1</id>
    <addr-items>
      <Addr-list>
        <addr>10.1.1.1/31</addr>
      </Addr-list>
    </addr-items>
  </If-list>
  <If-list>
    <id>eth1/2</id>
    <addr-items>
      <Addr-list>
        <addr>10.2.1.1/31</addr>
      </Addr-list>
    </addr-items>
  </If-list>
```

Phase 2: Configuring the device via NETCONF

Ansible

YANG model



NETCONF
Module

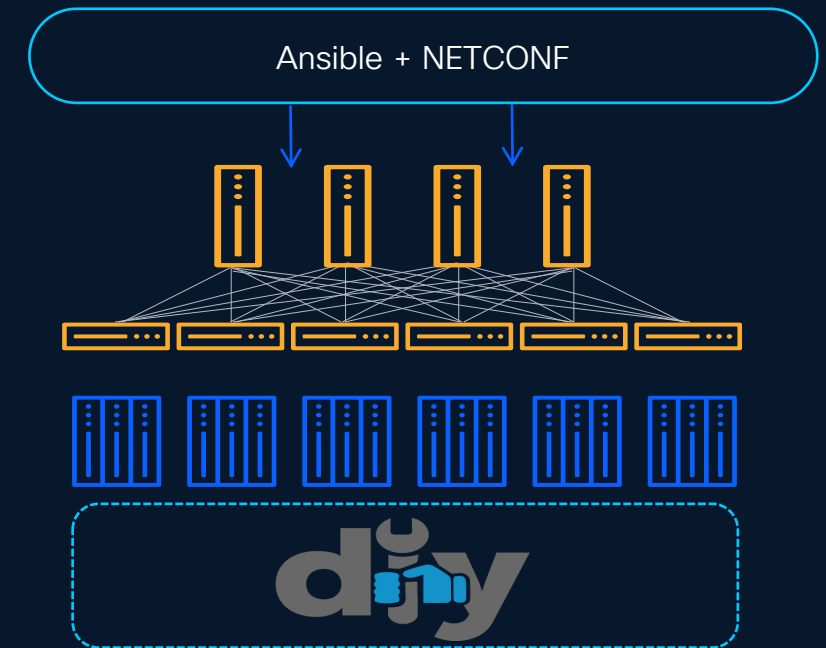


Network
Controller

```
<if-items>
  <If-list>
    <id>eth1/1</id>
    <addr-items>
      <Addr-list>
        <addr>10.1.1.1/31</addr>
      </Addr-list>
    </addr-items>
  </If-list>
  <If-list>
    <id>eth1/2</id>
    <addr-items>
      <Addr-list>
        <addr>10.2.1.1/31</addr>
      </Addr-list>
    </addr-items>
  </If-list>
```

```
<edit-config>
  <target>
    <candidate/>
  </target>
```

```
<commit>
  <confirmed/>
</commit>
```



Demo

Topology



Services: NTP and Syslogs



! deployServicesGenerateConfigs.yml M

! main.yml

NETCONF > 1. Services > NTP > roles > leaf > tasks > ! main.yml > {} 1 > name

Ansible Tasks Schema - Ansible tasks file (ansible.json)

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

- name: Build NETCONF NTP Configs

ansible.builtin.template:

src: "../jinja_templates/NETCONF/ntp.j2"

dest: "configs/NETCONF/{{inventory_hostname}}.xml"

delegate_to: localhost

- name: Build NETCONF NTP Filters

ansible.builtin.template:

src: "../jinja_templates/NETCONF/ntp_filter.j2"

dest: "filters/NETCONF/{{inventory_hostname}}.xml"

delegate_to: localhost

- name: Build NETCONF NTP test filter

ansible.builtin.template:

src: "../jinja_templates/NETCONF/ntp_test_filter.j2"

dest: "filters/NETCONF/check_ntp_peer_status_{{item.ip}}.xml"

loop: "{{ ntp_servers[0].servers }}"

delegate_to: localhost

run_once: true

! deployServices.yml M

NETCONF > 1. Services > ! deployServices.yml > {} 0 > [] tasks

2

9

59

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

- name: Apply NETCONF configuration

tasks:

- name: Assert that all devices are connected to NTP 1. Stratum 3+

ansible.builtin.assert:

- name: Ping Syslog server

ansible.builtin.shell:

gnocic -a {{inventory_hostname}}:57400 -u admin -p {{ ansible_password }} --skip-verify system

delegate_to: localhost

register: ping_response

- name: Debug Output

ansible.builtin.debug:

var: ping_response.stdout

- name: Assert that all devices have reachability to Syslog server

ansible.builtin.assert:

that: "'0.00% packet loss' in ping_response.stdout"

fail_msg: "Syslog Server is not reachable. Do not confirm the commit"

success_msg: "Syslog Server is reachable"

- name: Confirm commit if peer established

ansible.netcommon.netconf_config:

confirm_commit: true

- name: copy running to startup

ansible.netcommon.netconf_rpc:

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

bash - NETCONF

(ansible-venv) gsheehan@n9kdevnet:~/CiscoLive/devnet-2143/NETCONF\$ ansible-playbook -i ~/containerlab/devnet-2143_containerlab/clab-devnet-2143/ansible-inventory.yml 1.\ Services/deployServicesGenerateConfigs.yml

gsheehan@n9kdevnet:~/CiscoLive/devnet-2143/NETCONF\$ ssh spine1

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

spine1#

SSH: 172.22.30.195

main*

0 0 0

Gerard Sheehan (4 months ago)

Ln 9, Col 36

Spaces: 2

UTF-8

LF

YAML

Ansible Tasks Schema

Fabric: IP Addressing, Underlay and Overlay



! main.yml x ! leaf4.yml M

NETCONF > 2. deployFabric ~/CiscoLive/devnet-2143/NETCONF/2. deployFabric/Interfaces/host_vars/leaf4.yml

```
Ansible Tasks: Modified
1
2
3 - name: Build CLI Interface Configs
4   ansible.builtin.template:
5     src: "../jinja_templates/CLI/interfaces.j2"
6     dest: "configs/CLI/{{inventory_hostname}}.cfg"
7     delegate_to: localhost
8
9 - name: Build NETCONF Interfaces Configs
10  ansible.builtin.template:
11    src: "../jinja_templates/NETCONF/interfaces.j2"
12    dest: "configs/NETCONF/{{inventory_hostname}}.xml"
13    delegate_to: localhost
14
15 - name: Build NETCONF Interfaces Filters
16  ansible.builtin.template:
17    src: "../jinja_templates/NETCONF/interfaces_filter.j2"
18    dest: "filters/NETCONF/{{inventory_hostname}}.xml"
19    delegate_to: localhost
```

! deployFabric.yml M x

NETCONF > 2. deployFabric > ! deployFabric.yml > {} 0 > [] tasks > {} 1 > {} ansible.netcommon.netconf_config

```
2 - name: Apply NETCONF configuration
9   tasks:
46 - name: Configure Anycast MAC and NVE using NETCONF. Stage in candidate DB
47   ansible.netcommon.netconf_config:
49     content: "{{ lookup('file', './VXLAN/configs/NETCONF/{{inventory_hostname}}.xml') }}"
50     get_filter: "{{ lookup('file', './VXLAN/filters/NETCONF/{{inventory_hostname}}.xml') }}"
51     commit: False
52     lock: never
53     validate: True
54     when: inventory_hostname in groups['Leafs']
55
56 - name: Commit config with 45-second timeout
57   ansible.netcommon.netconf_config:
58     target: candidate
59     confirm: 45
60     confirm_commit: False
61     commit: True
62     lock: never
63
64 - name: Wait 30 seconds to allow for convergence
65   ansible.builtin.pause:
66     seconds: 30
67
68 - name: Get BGP peer state to spine1
69   ansible.netcommon.netconf_get:
70     filter: "{{ lookup('file', './BGP/filters/NETCONF/check_bgp_peer_status_spine1.xml') }}"
71     lock: never
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

ssh - NETCONF + - 🗑️ ... | 📄

(ansible-venv) gsheehan@n9kdevnet:~/CiscoLive/devnet-2143/NETCONF\$ ansible-playbook -i ~/containerlab/devnet-2143_containerlab/clab-devnet-2143/ansible-inventory.yml 2.\ deployFabric/deployFabric.yml

```
gsheehan@n9kdevnet:~/CiscoLive/devnet-2143/NETCONF$ ssh spine1
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
spine1#
```

vPC: vPC Domain and Port Channels



EXPLORER

DEVNET-2143 [SSH: 172.22.30.195]

ansible-venv

gNMI

NETCONF

.msgs

1. Services

2. deployFabric

3. vPC

4. addNetworks

ansible.cfg

.gitignore

config_vlan.py

leaf4.yml

main.yml M X

NETCONF > 3. vPC > vPCs > roles > leaf > tasks > ! main.yml > ...

Ansible Tasks Schema - Ansible tasks file (ansible.json)

1

2

3

4

5

6

7

8

9

10

11

12

13

- name: Build NETCONF vPC Configs

ansible.builtin.template:

src: "../jinja_templates/NETCONF/vpc.j2"

dest: "configs/NETCONF/{{inventory_hostname}}.xml"

delegate_to: localhost

- name: Build NETCONF vPC Filters

ansible.builtin.template:

src: "../jinja_templates/NETCONF/vpc_filter.j2"

dest: "filters/NETCONF/{{inventory_hostname}}.xml"

delegate_to: localhost

vPC.yml M X

NETCONF > 3. vPC > ! vPC.yml > {} 0 > [] tasks > {} 11 > {} ansible.netcommon.netconf_config > confirm_commit

2

9

51

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

- name: Apply NETCONF configuration

tasks:

- name: Get vPC Domain state

register: vpc_check

- name: Print return information from the previous task

ansible.builtin.debug:

var: vpc_check

when: inventory_hostname in groups['Leafs']

- name: Assert that all Leafs have vPC peering established

ansible.builtin.assert:

that: "<operSt>operational,peer-was-alive</operSt>" in vpc_check.stdout

fail_msg: "vPC is not Established. Do not confirm the commit"

success_msg: "vPC is Established"

when: inventory_hostname in groups['Leafs']

- name: Get Port-Channel status

ansible.netcommon.netconf_get:

filter: "{{ lookup('file', './vPCs/filters/NETCONF/check_port-channel_status.xml) }}"

lock: never

when: inventory_hostname in groups['Leafs']

register: pc_check

- name: Print return information from the previous task

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

(ansible-venv) gsheehan@n9kdevnet:~/CiscoLive/devnet-2143/NETCONF\$

ssh - NETCONF

gsheehan@n9kdevnet:~/CiscoLive/devnet-2143/NETCONF\$ ssh leaf1

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

leaf1#

SSH: 172.22.30.195

main*

0 0 0

Gerard Sheehan (4 months ago)

Ln 1, Col 4

Spaces: 2

UTF-8

LF

YAML

Ansible Tasks Schema

Conclusion

Complete your session surveys



Complete your surveys in the Cisco Events App.



Complete a minimum of 4 session surveys and the overall event survey to receive a unique Cisco Live t-shirt.

(from 11:30 on Thursday, while supplies last)

Continue your education



Visit the Cisco Showcase for related demos



Book your one-on-one Meet the Engineer meeting

Visit the Technical Solutions Clinics to discuss your technical questions



Attend the interactive education with DevNet, Capture the Flag, and Walk-in Labs



Visit the On-Demand Library for more sessions at CiscoLive.com/On-Demand

Thank you

